

MySQL - HackerRank Notebook

Pham Dang Hoang Thien

February 9, 2026

Student, Ho Chi Minh City, Vietnam
E-mail: hoangthien312006@gmail.com
HackerRank: hackerrank.com/artisthelios

Contents

1	Introduction to MySQL	3
1.1	SQL & Database là gì?	3
1.1.1	Database là gì?	3
1.1.2	SQL là gì?	3
1.1.3	Khái niệm bắt buộc phải nhớ	3
1.2	Thiết lập môi trường	3
1.3	Dữ liệu mẫu	4
1.4	Truy vấn cơ bản	5
1.4.1	Cấu trúc ”xương sống”	5
1.4.2	Ví dụ thực hành	5
1.4.3	Các syntax quan trọng	6
1.5	Xử lý dữ liệu & Hàm	6
1.6	Gom nhóm dữ liệu - GROUP BY	6
1.7	Nối bảng - JOIN	7
1.7.1	INNER JOIN - JOIN	7
1.7.2	LEFT JOIN	7
1.7.3	RIGHT JOIN	8
1.7.4	Tổng kết	8
2	Basic SQL	8
2.1	Triangle Classification Query	8
2.2	Revising Aggregations	10
2.2.1	The Count Function	10
2.2.2	The Sum Function	11
2.2.3	Averages	11
2.2.4	Average Population	12
2.2.5	Japan Population	12
2.2.6	Population Density Difference	13
2.3	The Blunder	13
2.4	Top Earners	15
2.5	Weather Observation	17
2.5.1	Station 2	18
2.5.2	Station 13	18
2.5.3	Station 14	19
2.5.4	Station 15	19
2.5.5	Station 16	19
2.5.6	Station 17	20
2.6	Population Census	20
2.7	African Cities	22

2.8	Average Population of Each Continent	22
2.9	Draw The Triangle 1	23
2.10	Draw The Triangle 2	24
3	Intermediate SQL	25
3.1	The PADS	25
3.2	Occupations	28
3.3	Binary Tree Nodes	29
3.4	New Companies	31
3.5	Print Prime Numbers*	32

1 Introduction to MySQL

1.1 SQL & Database là gì?

1.1.1 Database là gì?

Hãy tưởng tượng Database giống như một file Excel khổng lồ, nhưng thông minh hơn và chứa được hàng triệu dòng dữ liệu.

- Table (Bảng): Tương đương với một Sheet trong Excel.
- Row (Dòng/bản ghi): Một dòng dữ liệu (ví dụ: thông tin 1 khách hàng).
- Column (Cột/trường): Thuộc tính của dữ liệu (ví dụ: tên, tuổi, địa chỉ).

1.1.2 SQL là gì?

SQL (Structured QUery Language) là ngôn ngữ để bạn "nói chuyện" với Database. Bạn không dùng chuột kéo thả, mà dùng câu lệnh để yêu cầu Database trả về kết quả.

1.1.3 Khái niệm bắt buộc phải nhớ

- Primary Key (Khóa chính): Cột định danh duy nhất (ví dụ số CCCD, MSSV). Không được trùng, không được để trống.
- Foreign Key (Khóa ngoại): Cột dùng để nối sang bảng khác (ví dụ Trong bảng Đơn Hàng có cột Ma_Khach_Hang để biết ai mua).
- SQL không PHÂN BIỆT chữ HOA và chữ THƯỜNG.

1.2 Thiết lập môi trường

Để thực hành SQL, bạn cần cài đặt:

1. Cài đặt MySQL 8.0.45: MySQL Installer 8.0.45.
2. Cài đặt Workbench 8.0.45: MySQL Workbench 8.0.45.

Luyện tập trên HackerRank, LeetCode, v.v...

1.3 Dữ liệu mẫu

Cửa hàng trái cây Gồm 3 bảng:

1. KháchHang: Chứa thông tin người mua.
2. SanPham: Chứa thông tin hàng hóa.
3. DonHang: Lưu lịch sử giao dịch.

```
1  --Tao bang KháchHang
2  CREATE TABLE KháchHang(
3      ID INT PRIMARY KEY,
4      Ten TEXT,
5      ThanhPho TEXT
6  );
7  --Tao bang SanPham
8  CREATE TABLE SanPham(
9      ID INT PRIMARY KEY,
10     Ten TEXT,
11     Gia DECIMAL(10,2),
12     Loai TEXT
13 );
14 --Tao bang DonHang
15 CREATE TABLE DonHang(
16     ID INT PRIMARY KEY,
17     KháchHangID INT,
18     SanPhamID INT,
19     SoLuong INT,
20     NgayMua DATE
21 );
22 -- Them du lieu mau
23 INSERT INTO KháchHang VALUES
24 (1, 'Nguyen Van A', 'Ha Noi'),
25 (2, 'Tran Thi B', 'HCM'),
26 (3, 'Le Van C', 'Da Nang'),
27 (4, 'Pham Thi D', 'Ha Noi');
28
29 INSERT INTO SanPham VALUES
30 (101, 'Tao My', 50000, 'Trai Cay'),
31 (102, 'Banh My', 15000, 'Thuc Pham'),
32 (103, 'Nuoc Ngot', 10000, 'Do Uong'),
33 (104, 'Nho Han Quoc', 200000, 'Trai Cay');
34
```

```

35 INSERT INTO DonHang VALUES
36 (1, 1, 101, 2, '2023-10-01'), -- A mua 2 Tao
37 (2, 1, 103, 5, '2023-10-02'), -- A mua 5 Nuoc ngọt
38 (3, 2, 104, 1, '2023-10-03'), -- B mua 1 Nho
39 (4, 3, 102, 3, '2023-10-05'); -- C mua 3 Banh my
40 -- Khach D chua mua gi ca

```

1.4 Truy vấn cơ bản

1.4.1 Cấu trúc "xương sống"

SELECT (Cái gì) **FROM** (Ở đâu) **WHERE** (Điều kiện);

1.4.2 Ví dụ thực hành

Bài Toán 1: Lấy tên và thành phố của tất cả các khách hàng.

```

1 select Ten, ThanhPho
2 from KhachHang;

```

Bài Toán 2: Lấy tất cả thông tin của sản phẩm có giá trên 40,000VND.

```

1 Select * from SanPham
2 where Gia > 40000;

```

Note: Dấu * nghĩa là tất cả các cột. **Bài Toán 3 (kết hợp điều kiện):** Tìm sản phẩm là trái cây và giá dưới 100,000VND.

```

Select * from SanPham
where Loai = 'Trai Cay' and Gia < 100000;

```

1.4.3 Các syntax quan trọng

Câu lệnh	Ý nghĩa	Ví dụ
DISTINCT	Loại bỏ các giá trị trùng lặp	SELECT DISTINCT ThanhPho... (HN, HCM, ĐN 1 lần)
ORDER BY	Sắp xếp kết quả	ORDER BY Gia DESC (Giảm), ASC (Tăng)
LIMIT	Giới hạn số dòng kết quả	LIMIT 5 (5 dòng đầu)
LIKE	Tìm kiếm gần đúng	WHERE Ten LIKE 'N%' (tên bắt đầu N)
IN	Kiểm tra thuộc tập hợp	WHERE ThanhPho IN ('Ha Noi', 'HCM')

1.5 Xử lý dữ liệu & Hàm

Khi sếp hỏi: "Tổng doanh thu là bao nhiêu?", "Giá trung bình là bao nhiêu?", bạn không thể ngồi đếm từng dòng. **Các hàm phổ biến:**

1. COUNT(*): Đếm số dòng.
2. SUM(column): Tính tổng giá trị trong cột.
3. AVG(column): Tính giá trị trung bình.
4. MAX(column): Lấy giá trị lớn nhất trong cột.
5. MIN(column): Lấy giá trị nhỏ nhất trong cột.

Ví dụ: Đếm xem có bao nhiêu khách hàng ở Hà Nội.

```
1 Select count(*)
2 from KháchHang
3 where ThanhPho = 'Ha Noi';
```

→ Kết quả: 2

1.6 Gom nhóm dữ liệu - GROUP BY

Hãy tưởng tượng GROUP BY giống như Pivot Table trong Excel. Nó gom các dòng có giá trị giống nhau lại thành một nhóm để tính toán.

Quy Tắc Vàng: Nếu dùng hàm SUM, COUNT... đi kèm với một cột không

tính toán (như Tên, Thành Phố), bạn **BẮT BUỘC** phải **GROUP BY** cột đó.

Bài Toán: Mỗi Thành phố có bao nhiêu khách hàng?

```
1 Select ThanhPho, Count(*) as SoKhachHang
2 from KhachHang
3 group by ThanhPho;
```

→ Kết quả: Ha Noi - 2, HCM - 1, Da Nang - 1 **Bài Toán Nâng Cao:** Chỉ hiện thành phố nào có từ 2 khách trở lên.

```
1 Select ThanhPho, Count(*) as SoKhachHang
2 from KhachHang
3 group by ThanhPho
4 having SoKhachHang >= 2;
```

Lưu ý: **HAVING** giống **WHERE** nhưng dùng để lọc nhóm sau khi đã **GROUP BY**.

1.7 Nối bảng - JOIN

Dữ liệu thực tế nằm ở nhiều bảng. **JOIN** giúp bạn kết hợp dữ liệu từ nhiều bảng dựa trên một cột chung (thường là **Primary Key** và **Foreign Key**).

1.7.1 INNER JOIN - JOIN

Chỉ lấy những dòng có mặt ở cả 2 bảng.

Ví dụ: Chỉ hiện khách hàng đã từng mua hàng. Khách D chưa mua gì sẽ không hiện.

```
1 select k.Ten, d.NgayMua
2 from KhachHang k
3 INNER JOIN DonHang d --hoac JOIN cung duoc
4 ON k.ID = d.KhachHangID;
```

1.7.2 LEFT JOIN

Lấy tất cả bảng bên trái (**KhachHang**), bảng bên phải (**DonHang**) có thì hiện, không có thì để **NULL**.

Ví dụ: Danh sách tất cả khách hàng và ngày mua (nếu có).

```
1 select k.Ten, d.NgayMua
2 from KhachHang k
3 LEFT JOIN DonHang d
4 ON k.ID = d.KhachHangID;
```

→ Kết quả: Phạm Thị D sẽ hiện ra nhưng cột **NgayMua** là **NULL**.

1.7.3 RIGHT JOIN

Ngược lại với LEFT JOIN. Lấy tất cả bảng bên phải (đứng sau từ JOIN), bảng bên trái có thì hiện, không có thì NULL. **Tình huống:** Bạn muốn liệt kê tất cả Sản Phẩm trong kho, và xem nó đã được bán trong Đơn hàng nào. Nếu chưa bán thì hiện NULL.

Vì Bảng Trái: DonHang. Bảng Phải: SanPham - bảng được ưu tiên lấy hết.

```
1 SELECT DonHang.ID as MaDon, SanPham.TenSP
2 FROM DonHang
3 RIGHT JOIN SanPham ON DonHang.SanPhamID = SanPham.ID;
```

Thực tế: ít khi dùng RIGHT JOIN thay vào đó đổi vị trí 2 bảng và dùng LEFT JOIN cho dễ đọc từ trái sang phải.

DonHang RIGHT JOIN SanPham = SanPham LEFT JOIN DonHang

1.7.4 Tổng kết

Loại Join	Dữ liệu trả về	Khi nào dùng?
INNER JOIN	Chỉ phần chung của 2 bảng	Khi cần dữ liệu chắc chắn có ở 2 bảng
LEFT JOIN	Lấy cả bảng TRÁI & phần chung của bảng PHẢI (nếu không có thì NULL)	Khi muốn giữ toàn bộ danh sách gốc (VD: Danh sách lớp) dù họ có điểm thi hay chưa.
RIGHT JOIN	Lấy cả bảng PHẢI & phần chung của bảng TRÁI	Ít dùng. Dùng khi bảng quan trọng nằm phía sau câu lệnh.

2 Basic SQL

2.1 Triangle Classification Query

Write a query identifying the type of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral:** It's a triangle with 3 sides of equal length.
- **Isosceles:** It's a triangle with 2 sides of equal length.

- **Scalene:** It's a triangle with 3 sides of differing lengths.
- **Not A Triangle:** The given values of A, B, and C don't form a triangle.

Input Format:

<i>Column</i>	<i>Type</i>
<i>A</i>	<i>Integer</i>
<i>B</i>	<i>Integer</i>
<i>C</i>	<i>Integer</i>

The **TRIANGLES** table contains three columns: A, B, and C, representing the lengths of the sides of a triangle.

Sample Input:

A	B	C
20	20	23
20	20	20
20	21	22
13	14	30

Sample Output:

Isosceles
Equilateral
Scalene
Not A Triangle

Solution Hint

Use a **CASE** expression to classify each triangle in this order:

1. Check **Not A Triangle** first using the triangle inequality:

$$A + B \leq C \text{ or } A + C \leq B \text{ or } B + C \leq A$$

2. If valid, check **Equilateral**: $A = B$ AND $B = C$
3. Check **Isosceles**: exactly two sides are equal
4. Otherwise, it is **Scalene**

Syntax elements used:

- CASE WHEN ... THEN ... ELSE ... END
- Logical operators: AND, OR
- Comparison operators: =, <=

SQL Query

```
1 SELECT
2   CASE
3     WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A
4       Triangle'
5     WHEN A = B AND B = C THEN 'Equilateral'
6     WHEN A = B OR B = C OR A = C THEN 'Isosceles'
7     ELSE 'Scalene'
8   END AS triangle_type
FROM TRIANGLES;
```

Why this condition order?

- Put Not A Triangle first to filter out invalid side combinations.
- Then classify only valid triangles as equilateral, isosceles, or scalene.

2.2 Revising Aggregations

Input Format: The CITY table is described as follows:

<i>Fields</i>	<i>Type</i>
<i>ID</i>	<i>Number</i>
<i>NAME</i>	<i>VARCHAR2(17)</i>
<i>COUNTRYCODE</i>	<i>VARCHAR2(3)</i>
<i>DISTRICT</i>	<i>VARCHAR2(20)</i>
<i>POPULATION</i>	<i>Number</i>

2.2.1 The Count Function

Query a count of the number of cities in CITY having a Population larger than 100,000.

Solution Hint

The query should count the number of rows in the CITY table where the POPULATION is greater than 100,000.

COUNT(*) is used to count all rows that meet the specified condition in the WHERE clause. AS **city_count** renames the output column for clarity.

SQL Query

```
1 SELECT COUNT(*) AS city_count
2 FROM CITY
3 WHERE POPULATION > 100000;
```

2.2.2 The Sum Function

Query the total population of all cities in CITY where District is California.

Solution Hint

The query should sum the POPULATION column in the CITY table where the DISTRICT is 'California'.

SUM(POPULATION) is used to calculate the total population of rows that meet the specified condition in the WHERE clause. AS ***total_population*** renames the output column for clarity.

SQL Query

```
1 SELECT SUM(POPULATION) AS total_population
2 FROM CITY
3 WHERE DISTRICT = 'California';
```

2.2.3 Averages

Query the average population of all cities in CITY where District is California.

Solution Hint

The query should calculate the average of the POPULATION column in the CITY table where the DISTRICT is 'California'.

AVG(POPULATION) is used to calculate the average population of rows that meet the specified condition in the WHERE clause. AS ***average_population*** renames the output column for clarity.

SQL Query

```
1 SELECT AVG(POPULATION) AS average_population
2 FROM CITY
3 WHERE DISTRICT = 'California';
```

2.2.4 Average Population

Query the average population for all cities in CITY, rounded down to the nearest integer.

Solution Hint

The query should calculate the average of the POPULATION column in the CITY table and round it down to the nearest integer.

- Use **AVG(POPULATION)** to get the average population.
- Use **ROUND(...)** to round the average population to the nearest integer.
- AS **average_population** renames the output column for clarity.

SQL Query

```
1 SELECT ROUND(AVG(POPULATION)) AS average_population
2 FROM CITY;
```

2.2.5 Japan Population

Query the total population of cities in CITY where CountryCode is JPN.

Solution Hint

The query should sum the POPULATION column in the CITY table where the COUNTRYCODE is 'JPN'.

SUM(POPULATION) is used to calculate the total population of rows that meet the specified condition in the WHERE clause. AS **total_population** renames the output column for clarity.

SQL Query

```
1 SELECT SUM(POPULATION) AS total_population
2 FROM CITY
3 WHERE COUNTRYCODE = 'JPN';
```

2.2.6 Population Density Difference

Query the difference between the maximum and minimum populations in CITY.

Solution Hint

The query should calculate the difference between the maximum and minimum values of the POPULATION column in the CITY table.

- Use **MAX(POPULATION)** to get the maximum population.
- Use **MIN(POPULATION)** to get the minimum population.
- AS **pop_diff** renames the output column for clarity.

SQL Query

```
1 SELECT MAX(POPULATION) - MIN(POPULATION) AS pop_diff
2 FROM CITY;
```

2.3 The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the EMPLOYEES table, but did not realize her keyboard's 0 key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary. Write a query calculating the amount of error (i.e.: *actual-miscalculated* average monthly salaries), and round it up to the next integer. **Input Format**

The EMPLOYEES table is described as follows:

<i>Column</i>	<i>Type</i>
<i>ID</i>	<i>Integer</i>
<i>Name</i>	<i>String</i>
<i>Salary</i>	<i>Integer</i>

Note: Salary is per month.

Constraints

$$1000 < \text{Salary} < 10^5$$

Sample Input

<i>ID</i>	<i>Name</i>	<i>Salary</i>
1	Kristeen	1420
2	Ashley	2006
3	Julia	2210
4	Maria	3000

Sample Output

2061

Explanation

The table below shows the salaries without zeros as they were entered by Samantha:

<i>ID</i>	<i>Name</i>	<i>Salary</i>
1	Kristeen	142
2	Ashley	26
3	Julia	221
4	Maria	3

Samantha computes an average salary of 98.00. The actual average salary is 2159.00.

The resulting error between the two calculations is

$$2159.00 - 98.00 = 2061.00$$

Since it is equal to the integer 2061, it does not get rounded up.

Solution Hint

The query should calculate the difference between the actual average salary and the miscalculated average salary (with zeros removed). The difference should be rounded up to the next integer.

- ***AVG(Salary)*** is used to calculate the actual average salary.
- ***REPLACE(CAST(Salary AS CHAR), '0', '')*** is used to remove all zeros from the salary by converting it to a string, replacing '0' with an empty string, and then casting it back to an unsigned integer. /để hiểu là bỏ hết số 0 trong salary (mô phỏng bàn phím hỏng) đi rồi tính trung bình.
- ***AS UNSIGNED*** ensures that the modified salary is a positive integer for the average calculation. /nhận giá trị từ 0 trở lên.

SQL Query

```
1 SELECT CEIL(  
2     AVG(Salary) - AVG(CAST(REPLACE(CAST(Salary AS CHAR),  
3         '0', '') AS UNSIGNED))  
4 ) AS error  
FROM EMPLOYEES;
```

2.4 Top Earners

We define an employee's total earnings to be their monthly *salary* $\times$ *months* worked, and the maximum total earnings to be the maximum total earnings for any employee in the Employee table. Write a query to find the maximum total earnings for all employees as well as the total number of employees who have maximum total earnings. Then print these values as 2 space-separated integers. **Input Format:** The **EMPLOYEES** table containing employee data for a company is described as follows:

<i>Column</i>	<i>Type</i>
<i>employee_id</i>	<i>Integer</i>
<i>Name</i>	<i>String</i>
<i>Months</i>	<i>Integer</i>
<i>Salary</i>	<i>Integer</i>

where employee_id is an employee's ID number, name is their name, months is the total number of months they've been working for the company, and salary is their monthly salary.

Sample Input:

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

69952 1

Explanation

The table and earnings data are depicted below:

employee_id	name	months	salary	earnings
12228	Rose	15	1968	29520
33645	Angela	1	3443	3443
45692	Frank	17	1608	27336
56118	Patrick	7	1345	9415
59725	Lisa	11	2330	25630
74197	Kimberly	16	4372	69952
78454	Bonnie	8	1771	14168
83565	Michael	6	2017	12102
98607	Todd	5	3396	16980
99989	Joe	9	3573	32157

The maximum earnings value is 69952. The only employee with earnings = 69952 is Kimberly, so we print the maximum earnings value (69952) and the count of employees who earned \$69952 (1) as two space-separated values.

Solution Hint

eg. if total earnings is: 12000, 15000, 15000, 8000. Then the max total earnings is 15000, and there are 2 employees with that earning. So the output is "15000 2".

- 1. Calculate total earnings for each employee as *months * salary*.
- 2. Find the maximum total earnings using *MAX(months * salary)*.
- 3. Count the number of employees with total earnings equal to the maximum using *COUNT(*)* with a WHERE clause.

SQL Query

```

1 SELECT
2     MAX(months * salary) AS max_earnings,
3     COUNT(*) AS employee_count
4 FROM EMPLOYEES
5 WHERE (months * salary) = (
6     SELECT MAX(months * salary)
7     FROM EMPLOYEES
8 );

```

2.5 Weather Observation

Input Format: The STATION table is described as follows:

<i>Column</i>	<i>Type</i>
<i>ID</i>	<i>Number</i>
<i>CITY</i>	<i>VARCHAR2(21)</i>
<i>STATE</i>	<i>VARCHAR2(2)</i>
<i>LAT_N</i>	<i>Number</i>
<i>LONG_W</i>	<i>Number</i>

where LAT_N is the northern latitude and LONG_W is the western longitude.

2.5.1 Station 2

Query the following two values from STATION:

1. The sum of all values in LAT_N rounded to scale of 2 decimal places.
2. The sum of all values in LONG_W rounded to scale of 2 decimal places.

Output Format: Your results must be in the form:

lat lon

where *lat* is the sum of all values in LAT_N and *lon* is the sum of all values in LONG_W. Both results must be rounded to a scale of 2 decimal places.

Solution Hint

- Use **SUM(LAT_N)** to calculate the sum of all values in LAT_N.
- Use **SUM(LONG_W)** to calculate the sum of all values in LONG_W.
- Use **ROUND(..., 2)** to round both sums to a scale of 2 decimal places.
- AS **lat** and AS **lon** rename the output columns for clarity.

SQL Query

```
1 SELECT
2     ROUND(SUM(LAT_N), 2) AS lat,
3     ROUND(SUM(LONG_W), 2) AS lon
4 FROM STATION;
```

2.5.2 Station 13

Query the sum of Northern Latitudes (LAT_N) from STATION having values greater than 38.7880 and less than 137.2345. Truncate your answer to 4 decimal places.

SQL Query

```
1 SELECT ROUND(SUM(LAT_N), 4) AS total_latitude
2 FROM STATION
3 WHERE LAT_N > 38.7880 AND LAT_N < 137.2345;
```

2.5.3 Station 14

Query the greatest value of the Northern Latitudes (LAT_N) from STATION that is less than 137.2345. Truncate your answer to 4 decimal places.

Solution Hint

With x as a number and n as the number of decimal places:

- **ROUND**(x , n) is rounding function that rounds the number x to n decimal places. eg. **ROUND**(3.14159, 4) = 3.1416
- **TRUNCATE**(x , n) is truncating function that truncates the number x to n decimal places. eg. **TRUNCATE**(3.14159, 4) = 3.1415

SQL Query

```
1 SELECT TRUNCATE(MAX(LAT_N), 4) AS max_latitude
2 FROM STATION
3 WHERE LAT_N < 137.2345;
```

2.5.4 Station 15

Query the Western Longitude (LONG_W) for the largest Northern Latitude (LAT_N) in STATION that is less than 137.2345. Round your answer to 4 decimal places.

SQL Query

```
1 SELECT TRUNCATE(LONG_W, 4)
2 FROM STATION
3 WHERE LAT_N = (
4     SELECT MAX(LAT_N)
5     FROM STATION
6     WHERE LAT_N < 137.2345
7 );
```

2.5.5 Station 16

Query the smallest Northern Latitude (LAT_N) from STATION that is greater than 38.7780. Round your answer to 4 decimal places.

SQL Query

```
1 SELECT ROUND(MIN(LAT_N), 4)
2 FROM STATION
3 WHERE LAT_N > 38.7780;
```

2.5.6 Station 17

Query the Western Longitude (LONG_W) where the smallest Northern Latitude (LAT_N) in STATION is greater than 38.7780. Round your answer to 4 decimal places.

SQL Query

```
1 SELECT ROUND(LONG_W, 4) FROM STATION
2 WHERE LAT_N = (
3     SELECT MIN(LAT_N) FROM STATION
4     WHERE LAT_N > 38.7780
5 );
```

2.6 Population Census

Given the CITY and COUNTRY tables, query the sum of the populations of all cities where the CONTINENT is 'Asia'.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format:

<i>Column</i>	<i>Type</i>
<i>ID</i>	<i>NUMBER</i>
<i>NAME</i>	<i>VARCHAR2(17)</i>
<i>COUNTRYCODE</i>	<i>VARCHAR2(3)</i>
<i>DISTRICT</i>	<i>VARCHAR2(20)</i>
<i>POPULATION</i>	<i>NUMBER</i>

<i>Column</i>	<i>Type</i>
<i>CODE</i>	<i>VARCHAR2(3)</i>
<i>NAME</i>	<i>VARCHAR2(44)</i>
<i>CONTINENT</i>	<i>VARCHAR2(13)</i>
<i>REGION</i>	<i>VARCHAR2(25)</i>
<i>SURFACEAREA</i>	<i>NUMBER</i>
<i>INDEPYEAR</i>	<i>VARCHAR2(5)</i>
<i>POPULATION</i>	<i>NUMBER</i>
<i>LIFEEXPECTANCY</i>	<i>VARCHAR2(4)</i>
<i>GNP</i>	<i>NUMBER</i>
<i>GNPOLD</i>	<i>VARCHAR2(9)</i>
<i>LOCALNAME</i>	<i>VARCHAR2(44)</i>
<i>GOVERNMENTFORM</i>	<i>VARCHAR2(44)</i>
<i>HEADOFSTATE</i>	<i>VARCHAR2(32)</i>
<i>CAPITAL</i>	<i>VARCHAR2(4)</i>
<i>CODE2</i>	<i>VARCHAR2(2)</i>

Solution Hint

Join the CITY and COUNTRY tables on the matching key columns (CITY.CountryCode and COUNTRY.Code) and sum the populations of cities where the continent is 'Asia'.

- Use a **JOIN** clause to combine CITY and COUNTRY tables based on matching key columns.
- Use a **WHERE** clause to filter for rows where CO.CONTINENT = 'Asia'.

SQL Query

```

1 SELECT SUM(ci.population) AS total_population
2 FROM CITY ci
3 JOIN COUNTRY co
4   ON ci.CountryCode = co.Code
5 WHERE co.Continent = 'Asia';

```

2.7 African Cities

Given the CITY and COUNTRY tables, query the names of all cities where the CONTINENT is 'Africa'.

Note: the input format and table structures are the same as the previous problem.

SQL Query

```
1 SELECT ci.name AS city__Afr
2 FROM city ci
3 JOIN country co
4     ON ci.countrycode = co.code
5 WHERE co.continent = 'Africa';
```

2.8 Average Population of Each Continent

Given the CITY and COUNTRY tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

Note: the input format and table structures are the same as the previous problem.

Solution Hint

Using **GROUP BY clause**: is used to group rows that have **the same values** in specified columns into summary rows. It is often combined with aggregate functions like COUNT(), SUM(), AVG(), MAX(), and MIN() to produce summarized results./để yêu cầu tính trung bình dân số theo từng châu lục, nên ta sẽ nhóm các thành phố theo châu lục rồi tính trung bình dân số của từng nhóm đó.

SQL Query

```
1 SELECT co.continent, FLOOR(AVG(ci.population))
2 FROM CITY ci
3 JOIN COUNTRY co
4     ON ci.CountryCode = co.Code
5 GROUP BY co.continent;
```

2.9 Draw The Triangle 1

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern P(20).

Solution Hint

Use a recursive common table expression (CTE) to generate numbers from 20 down to 1, and then use the REPEAT function to create the desired pattern for each number.

- **WITH RECURSIVE** nums **AS** (...): This defines a recursive CTE named nums that generates numbers starting from 20 down to 1.
- **SELECT 20 AS n**: This is the base case of the recursion, starting with the number 20.
- **UNION ALL SELECT n - 1 FROM nums WHERE n > 1**: This is the recursive part that decrements n by 1 until it reaches 1.
- **SELECT REPEAT('*', n) AS pattern FROM nums**: This selects each number n from the CTE and uses the REPEAT function to generate a string of asterisks followed by spaces, repeated n times.

SQL Query

```
1 WITH RECURSIVE nums AS (  
2     SELECT 20 AS n  
3     UNION ALL  
4     SELECT n - 1  
5     FROM nums  
6     WHERE n > 1  
7 )  
8 SELECT REPEAT('*', n) AS pattern  
9 FROM nums;
```

2.10 Draw The Triangle 2

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Write a query to print the pattern P(20).

SQL Query

```
1 WITH RECURSIVE nums AS (  
2     SELECT 1 AS n  
3     UNION ALL  
4     SELECT n + 1  
5     FROM nums  
6     WHERE n < 20)  
7 SELECT REPEAT('*', n) AS pattern  
8 FROM nums;
```

3 Intermediate SQL

3.1 The PADS

Generate the following two result sets:

1. Query an alphabetically ordered list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).
2. Query the number of occurrences of each occupation in OCCUPATIONS. Sort the occurrences in ascending order, and output them in the following format:

There are a total of [occupation_count] [occupation]s.

where [occupation_count] is the number of occurrences of an occupation in OCCUPATIONS and [occupation] is the lowercase occupation name. If more than one Occupation has the same [occupation_count], they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation. **Input Format:** The OCCUPATIONS table is described as follows:

<i>Column</i>	<i>Type</i>
<i>name</i>	<i>String</i>
<i>occupation</i>	<i>String</i>

Occupations will only contain one of the following values: **Doctor, Professor, Singer, Actor**. **Sample Input:**

<i>name</i>	<i>occupation</i>
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output:

```

Ashely(P)
Christeen(P)
Jane(A)
Jenny(D)
Julia(A)
Ketty(P)
Maria(A)
Meera(S)
Priya(S)
Samantha(D)
There are a total of 2 doctors.
There are a total of 2 singers.
There are a total of 3 actors.
There are a total of 3 professors.

```

Explanation: The results of the first query are formatted to the problem description's specifications.

The results of the second query are ascendingly ordered first by number of names corresponding to each profession ($2 \leq 2 \leq 3 \leq 3$), and then alphabetically by profession (Doctor $\leq$ Singer, and actor $\leq$ Professor).

Solution Hint:

First Query: Print names with the first letter of their occupation in parentheses, ordered alphabetically.

eg. Ashley + Professor $\rightarrow$ Ashley(P)

Syntax elements used:

- `CONCAT(string1, string2, ...)`: Concatenates multiple strings into one.
- `LEFT(string, number)`: Extracts the leftmost 'number' characters from 'string'.
- `ORDER BY column`: Sorts the result set based on the specified column.

SQL Query - First Query

```
1 SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
2 FROM OCCUPATIONS
3 ORDER BY Name;
```

Second Query: Count people in each occupation and format the output as specified.

Syntax elements used:

- `COUNT(*)`: Counts the number of rows in each group.
- `LOWER(string)`: Converts 'string' to lowercase.
- `GROUP BY column`: Groups rows that have the same values in the specified column.
- `ORDER BY column1, column2`: Sorts the result set first by 'column1', then by 'column2'.
- `CONCAT(string1, string2, ...)`: Concatenates multiple strings into one./ghép câu

SQL Query - Second Query

```
1 SELECT CONCAT('There are a total of ', COUNT(*), ' ', LOWER(
   Occupation), 's.')
2 FROM OCCUPATIONS
3 GROUP BY Occupation --group by occupation to count each occupation
4 ORDER BY COUNT(*), Occupation; --sort by count asc, then occupation
   alphabetically
```

3.2 Occupations

Pivot the Occupation column in OCCUPATIONS so that each Name is sorted alphabetically and displayed underneath its corresponding Occupation. The output should consist of four columns (Doctor, Professor, Singer, and Actor) in that specific order, with their respective names listed alphabetically under each column.

Note: Print NULL when there are no more names corresponding to an occupation.

Note: Input format and Sample Input are the same as the previous problem.

Sample Output:

Jenny	Ashley	Meera	Jane
Samantha	Christeen	Priya	Julia
NULL	Ketty	NULL	Maria

Explanation: The first column is an alphabetically ordered list of Doctor names.

The second column is an alphabetically ordered list of Professor names.

The third column is an alphabetically ordered list of Singer names.

The fourth column is an alphabetically ordered list of Actor names.

The empty cell data for columns with less than the maximum number of names per occupation (in this case, the Professor and Actor columns) are filled with NULL values.

Solution Hint

OCCUPATIONS table has 2 columns: Name and Occupation. We need to pivot the Occupation column to create separate columns for each occupation type (Doctor, Professor, Singer, Actor) and list the corresponding names under each column.

Syntax elements used:

- **PARTITION BY column:** Divides the output into partitions to which the ROW_NUMBER() function is applied./ở đây ta chia bảng thành 4 nhóm nghề khác nhau.
- **ROW_NUMBER():** Assigns a unique sequential integer to rows within a partition of a result set, starting at 1 for the first row in each partition./đánh số thứ tự cho từng tên trong mỗi nhóm nghề.

- ***CASE WHEN condition THEN result END***: Evaluates a list of conditions like an IF-THEN-ELSE statement.
- ***GROUP BY column***: Groups rows that have the same values in the specified column.
- ***ORDER BY column***: Sorts the result set based on the specified column.

SQL Query:

```

1 SELECT
2     MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS
3         Doctor,
4     MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS
5         Professor,
6     MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS
7         Singer,
8     MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS
9         Actor
10 FROM (
11     SELECT
12         Name,
13         Occupation,
14         ROW_NUMBER() OVER (PARTITION BY Occupation ORDER
15             BY Name) AS rn
16     FROM OCCUPATIONS
17 ) ranked
18 GROUP BY rn
19 ORDER BY rn;

```

3.3 Binary Tree Nodes

You are given a table, BST, containing two columns: N and P, where N represents the value of a node in Binary Tree, and P is the parent of N.

<i>Column</i>	<i>Type</i>
<i>N</i>	<i>Integer</i>
<i>P</i>	<i>Integer</i>

Write a query to find the node type of Binary Tree ordered by the value of the node. Output one of the following for each node:

- **Root** If node is root node.
- **Leaf** If node is leaf node.
- **Inner** If node is neither root nor leaf node.

Sample Input:

N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	NULL

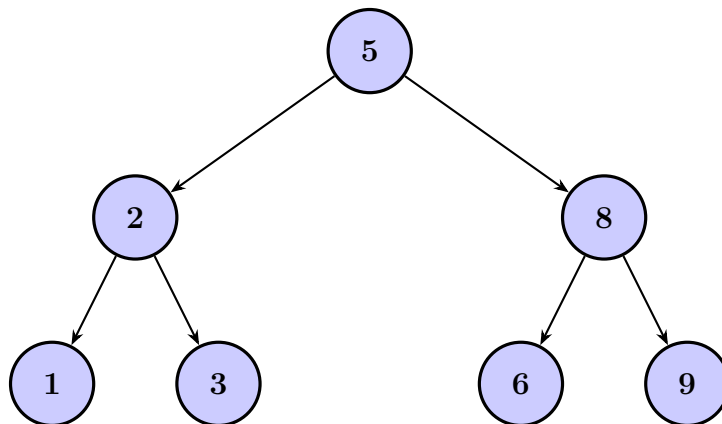
Sample Output:

```

1 Leaf
2 Inner
3 Leaf
5 Root
6 Leaf
8 Inner
9 Leaf

```

Explanation: The Binary Tree below illustrates the sample:



Solution Hint

To determine the type of each node in the binary tree, we can use conditional statements to check the values of N and P.

- If P is NULL, the node is the root.
- If N does not appear as a parent (P) for any other node, it is a leaf.
- Otherwise, it is an inner node.
- Use **CASE WHEN ... THEN ... ELSE ... END** to implement the conditional logic.

SQL Query:

```
1 SELECT
2     N,
3     CASE
4         WHEN P IS NULL THEN 'Root'
5         WHEN N NOT IN (SELECT P FROM BST WHERE P IS NOT
6             NULL) THEN 'Leaf'
7         ELSE 'Inner'
8     END
9 FROM BST
ORDER BY N;
```

3.4 New Companies

Problem Statement: HackerRank - The Company

Solution Hint

Connect the Company, Lead_Manager, Senior_Manager, Manager, and Employee tables using JOINS based on their respective foreign key relationships. Then, use COUNT(DISTINCT ...) to count the unique lead managers, senior managers, managers, and employees for each company. Finally, group the results by company code and founder.

- Use **JOIN** clauses to connect the tables based on their relationships.
- Use **COUNT(DISTINCT column)** to count unique entries in each category./đếm số mã khác nhau (không trùng) vì khi join có thể bị lặp lại.

- Eg. 1 manager, 3 employees → khi join sẽ ra 3 mã manager giống nhau, nên ta dùng DISTINCT để chỉ đếm 1 mã manager đó thôi.
- Use **GROUP BY** to aggregate results by company code and founder.

SQL Query

```

1 SELECT
2     c.company_code,
3     c.founder,
4     COUNT(DISTINCT lm.lead_manager_code) AS total_lead_managers,
5     COUNT(DISTINCT sm.senior_manager_code) AS
6         total_senior_managers,
7     COUNT(DISTINCT m.manager_code) AS total_managers,
8     COUNT(DISTINCT e.employee_code) AS total_employees
9 FROM Company c
10 JOIN Lead_Manager lm
11     ON c.company_code = lm.company_code
12 JOIN Senior_Manager sm
13     ON lm.lead_manager_code = sm.lead_manager_code
14 JOIN Manager m
15     ON sm.senior_manager_code = m.senior_manager_code
16 JOIN Employee e
17     ON m.manager_code = e.manager_code
18 GROUP BY
19     c.company_code,
20     c.founder

```

3.5 Print Prime Numbers*

Write a query to print all prime numbers less than or equal to 1000. Print your result on a single line, and use the ampersand and character as your separator (instead of a space).

For example, the output for all prime numbers ≤ 10 would be:

2&3&5&7

Solution Hint

To find all prime numbers less than or equal to 1000, we can use a recursive common table expression (CTE) to generate numbers from 2 to 1000. Then, we can filter out the non-prime numbers by checking for divisibility.

- Use a recursive CTE to generate numbers from 2 to 1000.
- Use a ***WHERE NOT EXISTS*** clause to filter out non-prime numbers by checking if there are any divisors for each number.
- Use ***GROUP_CONCAT(... SEPARATOR '&')*** to concatenate the prime numbers into a single string with '&' as the separator.
- Use ***FLOOR(SQRT(nums.n))*** to limit the divisor check up to the square root of the number./nghĩa là check từ 2 đến $\sqrt{n}$.
- Check divisibility using ***nums.n % d.n = 0***.

SQL Query

```

1 WITH RECURSIVE nums AS (
2     SELECT 2 AS n
3     UNION ALL
4     SELECT n + 1
5     FROM nums
6     WHERE n < 1000
7 )
8 SELECT GROUP_CONCAT(n ORDER BY n SEPARATOR '&') AS
   primes
9 FROM nums
10 WHERE NOT EXISTS (
11     SELECT 1
12     FROM nums d
13     WHERE d.n <= FLOOR(SQRT(nums.n))
14     AND nums.n % d.n = 0
15     AND d.n > 1
16     AND d.n < nums.n
17 );

```